PYTHON PROGRAMMING

ZERO TO PRO

A Practical, Hands-On Training Manual for Complete Beginners

Training Note & Course Guide

Learn by Doing | Every Lesson Includes Working Code You Type and Run Yourself

Offered by

BRIGHT EMERALD COMPUTER ACADEMY

Bright Emerald
Computer Academy
Rooted in Knowledge, Growing in Ianovation

Trainer / Tutor: Emmanuel Sebit Joseph

TABLE OF CONTENT

How to use this Guide

This is a practical, project-based training manual. It is built for people who have never written a line of code before, and it takes you all the way to writing real, working Python programs with confidence.

The Learning Method

- Read the concept — short, plain-English explanations, no jargon left unexplained.
- See the code — every idea is shown with a real, runnable example.
- Type it yourself — never copy-paste. Typing builds muscle memory.
- Do the practice task — each lesson ends with a "Try It Yourself" exercise.
- Build the module project — every module ends with a small project combining everything learned.

> **TIP**
> Keep a single folder called python practice on your computer. Save every exercise as its own file, e.g. lesson_3.py. In six weeks, you'll have a portfolio of your own progress.

What You Need Before You Start

Requirement	Details
Computer	Windows, macOS, or Linux — any modern laptop or desktop works
Software	Python 3.12+ (free) and a code editor such as VS Code (free)
Internet	Needed only for installation and occasional lookups
Prior experience	None. This guide assumes zero coding background
Time commitment	45–90 minutes per lesson, 3–5 lessons per week recommended

Course Description

This course takes an absolute beginner from installing Python for the first time to writing complete, working programs — including advanced data analysis, web development, array manipulation, object-oriented design, and real-world automation projects. The approach is deliberately practical: every concept is taught through code you write and run yourself, not just theory you read. By the end, you will have a portfolio of 20+ working programs and several complete capstone projects.

This training is offered by Bright Emerald Computer Academy, delivered by Trainer/Tutor Emmanuel Sebit Joseph.

Course Outline

Module	Title	What You'll Be Able To Do
a	Setup & Orientation	Install Python, run your first program, navigate the editor
1	Python Foundations	Variables, data types, input/output, basic arithmetic
2	Making Decisions	if/elif/else logic, comparison & boolean operators
3	Repeating Actions	for and while loops, break/continue, nested loops
4	Data Structures	Lists, tuples, dictionaries, sets — store & organize data
5	Functions	Write reusable, organized code with parameters & return values
6	Strings & Text Processing	Slice, search, format, and manipulate text like a pro
7	Files & Error Handling	Read/write files, handle errors gracefully with try/except
8	Object-Oriented Programming	Classes, objects, inheritance — model real-world things
9	Modules, Libraries & Packages	Use pip, import libraries, structure multi-file projects
10	Working With Real Data	CSV/JSON handling, intro to APIs and requests
11	Intro to Popular Libraries	NumPy, pandas basics — a taste of data & automation work
12	Arrays & Array Arrangement	Build, reshape, sort, search, and stack arrays confidently
13	Advanced Data Analysis	Clean, group, merge, visualize real datasets with pandas
14	Web Development with Python	Build and run a working website using Flask
15	Simple Projects Practice Bank	20 short project ideas to practice every topic covered
16	Capstone Projects	Build 5 complete portfolio projects from scratch
17	Pro Habits & Next Steps	Debugging, testing, Git basics, essential commands, and where to go from here

Course Goals

- Understand core programming logic; not just Python syntax, but how programmers think.
- Write clean, readable, well-organized code following real industry conventions.
- Be able to independently debug and fix your own code when it breaks.
- Build complete small programs and automation scripts without hand-holding.
- Have a working foundation to move into web development, data analysis, or automation.

Setup & Orientation

Duration	1 lesson, ~45 minutes
Goal	Get Python installed and run your first program

Installing Python

1. Go to python.org/downloads and download the latest Python 3 version for your operating system.
2. Run the installer. On Windows, tick the box that says "Add Python to PATH" before clicking Install, this step is easy to miss and causes most beginner setup problems.
3. Open your terminal (Command Prompt on Windows, Terminal on macOS/Linux) and type: python --version (or python3 --version on Mac/Linux).
4. If you see a version number like Python 3.12.4, installation succeeded.

> **⚠ COMMON MISTAKE**
> If your terminal says "python is not recognized," it usually means PATH wasn't set during install. Reinstall and make sure the PATH checkbox is ticked, or search "add Python to PATH [your OS]" for a quick fix.

Installing a Code Editor

We recommend Visual Studio Code (VS Code); free, beginner-friendly, and used widely in the industry.

1. Download it from code.visualstudio.com and install it.
2. Open VS Code, go to the Extensions panel (icon on the left sidebar), search "Python," and install the official Microsoft Python extension.
3. Create a new folder called python practice anywhere on your computer; this is your workspace for the entire course.

Your First Program

In VS Code, open your python_practice folder, create a new file named hello.py, and type the following exactly:

```
print("Hello, World!")
```

Run it: open the terminal inside VS Code (Terminal menu → New Terminal) and type:

```
python hello.py
```

You should see Hello, World! printed in the terminal. Congratulations you're officially a programmer.

> **TIP**
> print() is a function that displays output on screen. You will use it constantly including to debug your code by checking what values variables hold.

> **🛠 TRY IT YOURSELF**
> Change the message inside the quotes to something about yourself, e.g. print("My name is Sarah and I'm learning Python"). Run it again.

Module 1 Python Foundations

Duration	4 lessons, ~4 hours total
Goal	Understand variables, data types, and basic operations

Lesson 1.1 Variables

A variable is a named container that stores a value. Think of it as a labeled box you can put things into and take things out of.

```
name = "Amina"
age = 25
height = 1.68
print(name)
print(age)
```

- Variable names should be lowercase, descriptive, and use underscores for spaces: first_name, not FirstName or fn.
- Names cannot start with a number and cannot contain spaces or symbols like - or @.
- Python is case-sensitive: age and Age are two different variables.

> **🛠 TRY IT YOURSELF**
> Create three variables: your name, your age, and your favorite hobby. Print all three using separate print() statements.

Lesson 1.2 Core Data Types

Every value in Python has a type. The four you'll use constantly:

Type	Example
int	25
float	1.68
str	"Amina"
bool	True / False

Check any value's type using the built-in type() function:

```
print(type(25))      # <class 'int'>
print(type("hi"))    # <class 'str'>
print(type(3.14))    # <class 'float'>
```

> **TIP**
> When something breaks unexpectedly, print(type(your_variable)) is one of the fastest ways to find out what's actually going on.

Lesson 1.3 Basic Math & Operators

```
a = 10
b = 3
print(a + b)   # 13  addition
print(a - b)   # 7   subtraction
print(a * b)   # 30  multiplication
print(a / b)   # 3.333...  division (always gives a float)
print(a // b)  # 3   floor division (drops the decimal)
print(a % b)   # 1   modulus (remainder)
print(a ** b)  # 1000  exponent (power)
```

> **⚠ COMMON MISTAKE**
> A very common beginner mix-up: / always returns a float (even 10 / 2 gives 5.0), while // gives a whole number by dropping everything after the decimal.

Lesson 1.4 Getting Input & String Formatting

Programs become interactive with the input() function, which pauses and waits for the user to type something:

```
user_name = input("What is your name? ")
user_age = input("How old are you? ")
print(f"Hello {user_name}, you are {user_age} years old.")
```

The f before the quotation mark makes it an f-string, the cleanest, most modern way to insert variables into text. Anything inside curly braces { } gets evaluated and inserted.

> **⚠ COMMON MISTAKE**
> input() always returns text (a string), even if the user types a number. To do math with it, convert first: age = int(input("Age: ")).

> **🛠 TRY IT YOURSELF**
> Write a small program that asks the user for their name and favorite number, then prints: "{name}, your favorite number times 10 is {result}". Remember to convert the number input using int().

Module 1 Mini-Project: Personal Info Card

Build a program that:

1. Asks the user for their name, age, city, and one hobby.
2. Calculates what year they were born (assume current year 2026).
3. Prints a neatly formatted "info card" using an f-string, e.g.:

```
=== INFO CARD ===
Name: Amina
Age: 25 (Born around 2001)
City: Kampala
Hobby: Reading
```

Module 2 Making Decisions

Duration	2 lessons, ~2 hours
Goal	Control what your program does based on conditions

Lesson 2.1 Comparison & Boolean Operators

```
print(5 > 3)     # True
print(5 == 5)    # True (equality check, two equals signs)
print(5 != 3)    # True (not equal)
print(5 >= 5)    # True
print(3 < 2)     # False
```

Combine conditions with and, or, and not:

```
age = 20
has_id = True
print(age >= 18 and has_id)   # True — both must be true
print(age >= 18 or has_id)    # True — at least one is true
print(not has_id)             # False — flips the value
```

> **⚠ COMMON MISTAKE**
> Using a single = instead of == is one of the most common beginner errors. = assigns a value; == checks equality.

Lesson 2.2 if, elif, else

```
age = int(input("Enter your age: "))

if age < 13:
    print("You are a child.")
elif age < 20:
    print("You are a teenager.")
elif age < 65:
    print("You are an adult.")
else:
    print("You are a senior.")
```

- Indentation is not optional in Python — it defines which code belongs inside the if block. Always use 4 spaces (VS Code does this automatically).
- elif means "else if" check another condition only if the first was false.
- else catches everything that didn't match any condition above it.

> **TIP**
> Python checks conditions top to bottom and stops at the first true one. Order matters, put the most specific conditions first.

> **TRY IT YOURSELF**
> Write a program that asks for a number and prints whether it's positive, negative, or zero.

Module 2 Mini-Project: Simple Grade Calculator

Ask the user for a score out of 100 and print their letter grade using this scale:

90–100	**A**
80–89	B
70–79	C
60–69	D
Below 60	F

Module 3 Repeating Actions (Loops)

Duration	3 lessons, ~3 hours
Goal	Automate repetitive tasks with for and while loops

Lesson 3.1 for Loops

A for loop repeats an action for each item in a sequence, most commonly, a range of numbers:

```
for i in range(5):
    print(i)
# Output: 0 1 2 3 4 (range starts at 0, stops before 5)
```

```
for i in range(1, 6):
    print(f"Count: {i}")
# Output: Count: 1 through Count: 5
```

You can also loop directly over text or a list of items:

```
for letter in "Python":
    print(letter)
```

> **TIP**
> range(start, stop, step) has a third optional argument. range(0, 10, 2) counts by 2s: 0, 2, 4, 6, 8.

Lesson 3.2 while Loops

A while loop repeats as long as a condition stays true. Use it when you don't know in advance how many times you'll repeat.

```
count = 0
while count < 5:
   print(f"Count is {count}")
   count += 1  # same as count = count + 1
```

> **⚠ COMMON MISTAKE**
> If you forget to change the condition inside the loop (like count += 1 above), the loop runs forever. This is called an infinite loop — press Ctrl+C in the terminal to stop it.

Lesson 3.3 break, continue & Nested Loops

```
for i in range(10):
    if i == 5:
        break       # stops the loop completely
    print(i)
```

```
for i in range(10):
    if i % 2 == 0:
        continue    # skips to the next iteration
    print(i)          # only prints odd numbers
```

Loops can be placed inside other loops; useful for grids, tables, and combinations:

```
for row in range(3):
    for col in range(3):
        print(f"({row},{col})", end=" ")
    print()   # new line after each row
```

> **🛠 TRY IT YOURSELF**
> Write a program that prints all numbers from 1 to 50 that are divisible by 3, using a for loop and the % operator.

Module 3 Mini-Project: Number Guessing Game

Build a game that:

1. Picks a "secret number" (hardcode it, e.g. 7, for now; random numbers come in Module 9).
2. Uses a while loop to keep asking the user to guess until they get it right.
3. Tells them "Too high" or "Too low" after each wrong guess, and "Correct!" when they win, along with how many guesses it took.

Module 4 Data Structures

Duration	4 lessons, ~4 hours
Goal	Store and organize collections of data

Lesson 4.1 Lists

A list holds an ordered collection of items, and it can change (be added to, removed from, edited) after creation.

```
fruits = ["apple", "banana", "mango"]
print(fruits[0])        # apple  (indexing starts at 0)
print(fruits[-1])       # mango  (negative index counts from the end)
fruits.append("grape")  # adds to the end
fruits.remove("banana") # removes by value
print(len(fruits))      # 3  — number of items
```

Loop through a list to process every item:

```
for fruit in fruits:
    print(f"I like {fruit}")
```

> **TIP**
> List slicing: fruits[0:2] gives items from index 0 up to (not including) index 2. fruits[:2] and fruits[1:] work as shortcuts for "from the start" and "to the end."

Lesson 4.2 Tuples

A tuple is like a list, but immutable — once created, it cannot be changed. Use it for fixed data, like coordinates.

```
point = (4, 7)
print(point[0])   # 4
# point[0] = 9   # this would cause an error — tuples can't be edited
```

Lesson 4.3 Dictionaries

A dictionary stores key-value pairs; perfect for representing real-world records:

```
person = {
    "name": "Amina",
    "age": 25,
    "city": "Kampala"
}
print(person["name"])       # Amina
person["age"] = 26          # update a value
person["job"] = "Engineer"  # add a new key

for key, value in person.items():
    print(f"{key}: {value}")
```

> **⚠ COMMON MISTAKE**
> Accessing a key that doesn't exist (person["salary"]) causes an error. Use person.get("salary", "Not found") to safely handle missing keys instead.

Lesson 4.4 Sets

A set stores unique, unordered items; duplicates are automatically removed. Useful for removing repeats or checking membership fast.

```
numbers = [1, 2, 2, 3, 3, 3, 4]
unique_numbers = set(numbers)
print(unique_numbers)   # {1, 2, 3, 4}
```

> **🛠 TRY IT YOURSELF**
> Create a dictionary representing a product (name, price, quantity). Print a sentence like "5 x Notebook @ $2.50 = $12.50" by calculating price × quantity.

Module 4 Mini-Project: Contact Book

Build a program that stores contacts in a list of dictionaries, e.g. {"name": "John", "phone": "0700123456"}. It should:

1. Let the user add a new contact (name + phone).
2. Print all saved contacts in a numbered, readable list.
3. Let the user search for a contact by name.

Module 5 — Functions

Duration	3 lessons, ~3 hours
Goal	Write clean, reusable, organized code

Lesson 5.1 Defining & Calling Functions

A function is a named, reusable block of code. Define it once with def, then call it as many times as needed.

```
def greet():
    print("Hello there!")

greet()   # calling the function runs the code inside it
greet()   # can be called again and again
```

Lesson 5.2 Parameters, Arguments & Return Values

```
def greet(name):                  # "name" is a parameter
    print(f"Hello, {name}!")

greet("Amina")                    # "Amina" is the argument
greet("John")
```

Functions can send a value back using return, so the result can be stored or reused:

```
def add(a, b):
    return a + b

result = add(4, 7)
print(result)   # 11
```

> **⚠ COMMON MISTAKE**
> print() displays something on screen but gives nothing back to the program. return actually hands a value back so you can use it elsewhere. Mixing these up is a very common beginner error.

Lesson 5.3 Default Values & Keyword Arguments

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Amina")                    # Hello, Amina!
greet("John", "Good morning")     # Good morning, John!
greet(name="Sarah", greeting="Hi")  # keyword arguments — order doesn't matter
```

> **TIP**
> Good function names describe an action: calculate_total(), send_email(), get_average(). If you can't name it clearly, the function is probably trying to do too much.

> **TRY IT YOURSELF**
> Write a function calculate_area(length, width) that returns the area of a rectangle. Call it three times with different values and print each result.

Module 5 Mini-Project: Simple Calculator

Build a calculator using separate functions: add(a, b), subtract(a, b), multiply(a, b), divide(a, b). The program should:

1. Ask the user to choose an operation and enter two numbers.
2. Call the matching function and print the result.
3. Handle division by zero gracefully (check before dividing, print a friendly message instead of crashing).

Module 6 Strings & Text Processing

Duration	2 lessons, ~2 hours
Goal	Manipulate and analyze text confidently

Lesson 6.1 String Basics & Slicing

```
text = "Python Programming"
print(text[0])          # P
print(text[0:6])        # Python
print(text.upper())     # PYTHON PROGRAMMING
print(text.lower())     # python programming
print(len(text))        # 19
print(text.replace("Python", "Java"))  # Java Programming
```

Lesson 6.2 Useful String Methods

Method	What It Does
.strip()	Removes extra spaces from the start and end
.split(',')	Breaks text into a list using a separator
.join(list)	Combines a list of strings into one, using a separator
.find('x')	Returns the position of 'x', or -1 if not found
.startswith('x')	Returns True/False
.isdigit()	Returns True if the text is all numbers

```
sentence = "  banana,apple,mango  "
clean = sentence.strip()
fruits = clean.split(",")
print(fruits)                  # ['banana', 'apple', 'mango']
print(", ".join(fruits))       # banana, apple, mango
```

> **TIP**
> Combine .strip() and .split() together constantly when cleaning up messy user input or data from files.

> **TRY IT YOURSELF**
> Ask the user to type a sentence. Print how many words it contains, and print it with every word capitalized (hint: research the .title() method).

Module 7 Files & Error Handling

Duration	3 lessons, ~3 hours
Goal	Save/load data and handle errors without crashing

Lesson 7.1 Reading & Writing Files

```
# Writing to a file
with open("notes.txt", "w") as file:
    file.write("This is my first saved file.\n")
    file.write("Second line here.")
```

```
# Reading a file
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
```

- "w" mode overwrites the file completely — use "a" (append) to add to the end without deleting existing content.
- The with statement automatically closes the file when done, even if an error occurs. Always prefer it over open()/close() manually.

Lesson 7.2 Reading Line by Line

```
with open("notes.txt", "r") as file:
    for line in file:
        print(line.strip())   # .strip() removes the trailing newline
```

Lesson 7.3 try/except: Handling Errors Gracefully

Errors (called exceptions) crash your program unless you handle them. try/except lets you catch problems and respond instead of crashing:

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(result)
except ValueError:
    print("That's not a valid number.")
except ZeroDivisionError:
    print("You can't divide by zero.")
```

> **TIP**
> Catch specific error types (ValueError, ZeroDivisionError, FileNotFoundError) rather than a bare except: — it makes bugs far easier to diagnose.

> **🛠 TRY IT YOURSELF**
> Write a program that tries to open a file called "data.txt". If the file doesn't exist, catch the FileNotFoundError and print a friendly message instead of crashing.

Module 7 Mini-Project: Journal App

Build a simple command-line journal that:

1. Lets the user type a journal entry.
2. Appends it to a file called journal.txt, with the entry on its own line.
3. Has a menu: 1) Add entry 2) View all entries 3) Exit — using a while loop and if/elif.
4. Handles the case where journal.txt doesn't exist yet without crashing.

Module 8 Object-Oriented Programming (OOP)

Duration	4 lessons, ~4 hours
Goal	Model real-world things using classes and objects

Lesson 8.1 Classes & Objects

A class is a blueprint for creating objects. An object is a specific instance built from that blueprint — like how "Car" is a concept, but "my red Toyota" is an actual object.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

my_dog = Dog("Rex", 3)
my_dog.bark()          # Rex says Woof!
print(my_dog.age)      # 3
```

- __init__ is the constructor — it runs automatically when a new object is created, setting up its starting values.
- self refers to the specific object being worked with. It must be the first parameter of every method.

Lesson 8.2 Multiple Objects & Methods

```
dog1 = Dog("Rex", 3)
dog2 = Dog("Bella", 5)
dog1.bark()   # Rex says Woof!
dog2.bark()   # Bella says Woof!
# Each object keeps its own separate data
```

Lesson 8.3 Inheritance

A class can inherit from another class, reusing its code and adding or overriding behavior:

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound.")

class Cat(Animal):          # Cat inherits from Animal
    def speak(self):        # overrides the parent's method
        print(f"{self.name} says Meow!")

generic = Animal("Creature")
generic.speak()             # Creature makes a sound.
cat = Cat("Whiskers")
cat.speak()                 # Whiskers says Meow!
```

> **TIP**
> Use OOP when you're modeling things with both data and behavior — users, bank accounts, game characters, products. For simple one-off scripts, plain functions are often enough.

> **TRY IT YOURSELF**
> Create a class BankAccount with attributes owner and balance, and methods deposit(amount) and withdraw(amount) that update the balance. Print the balance after a few transactions.

Module 8 Mini-Project: Library System

Build a small system using classes:

1. A Book class with title, author, and is_checked_out (boolean).
2. A Library class that holds a list of Book objects, with methods to add_book(), checkout_book(title), and return_book(title).
3. A method to display_available_books() that lists only books not checked out.

Module 9 Modules, Libraries & Packages

Duration	2 lessons, ~2 hours
Goal	Use Python's built-in tools and install external libraries

Lesson 9.1 Importing Built-In Modules

Python comes with many built-in modules ready to use — no installation needed:

```
import random
import math
import datetime

print(random.randint(1, 10))     # random whole number between 1 and 10
print(math.sqrt(16))             # 4.0
print(datetime.date.today())     # today's date
```

You can also import just one function from a module:

```
from random import choice
options = ["rock", "paper", "scissors"]
print(choice(options))
```

Lesson 9.2 Installing External Packages with pip

pip is Python's package installer, used to add external libraries not built into Python. Run this in your terminal (not inside a .py file):

```
pip install requests
```

Then use it in your code like any other module:

```
import requests
response = requests.get("https://api.github.com")
print(response.status_code)   # 200 means success
```

> **TIP**
> If pip doesn't work, try pip3 install requests or python -m pip install requests — naming conventions vary slightly by operating system.

> **TRY IT YOURSELF**
> Update your Number Guessing Game from Module 3 to use random.randint(1, 100) to pick the secret number instead of a hardcoded one.

Module 10 Working with Real Data

Duration	3 lessons, ~3 hours
Goal	Handle CSV/JSON data and call a real API

Lesson 10.1 Working with CSV Files

```
import csv

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], row["score"])
```

```
# Writing a CSV
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "score"])
    writer.writerow(["Amina", 92])
    writer.writerow(["John", 85])
```

Lesson 10.2 Working With JSON

JSON is the standard format for exchanging structured data between programs and APIs. It maps almost directly onto Python dictionaries and lists.

```
import json

data = {"name": "Amina", "age": 25, "skills": ["Python", "SQL"]}

# Convert Python object to a JSON string and save it
with open("profile.json", "w") as file:
    json.dump(data, file)

# Load it back
with open("profile.json", "r") as file:
    loaded = json.load(file)
print(loaded["name"])
```

Lesson 10.3 Calling a Real API

An API lets your program request live data from the internet. Using the requests library from Module 9:

```
import requests

response = requests.get("https://api.github.com/users/octocat")
data = response.json()      # converts the response directly to a dict
print(data["login"])
print(data["public_repos"])
```

> **⚠ COMMON MISTAKE**
> Always check response.status_code before trusting the data — 200 means success, 404 means not found, 500 means server error.

> **🛠 TRY IT YOURSELF**
> Use requests to call https://api.github.com/users/ followed by any GitHub username, and print their number of followers and public repos.

Module 10 Mini-Project: Expense Tracker

Build a program that:

1. Let the user add expenses (description, amount, category) and saves them to a JSON file.
2. Loads existing expenses on startup so nothing is lost between runs.
3. Calculates and prints the total spent, and the total per category.

Module 11 Intro to Popular Libraries (NumPy & pandas)

Duration	2 lessons, ~2.5 hours
Goal	Get a practical first taste of data-focused Python

Lesson 11.1 NumPy Basics

NumPy is the foundation of numerical computing in Python — fast arrays and math operations. Install with pip install numpy.

```
import numpy as np

scores = np.array([88, 92, 79, 95, 60])
print(scores.mean())     # average
print(scores.max())      # highest value
print(scores.min())      # lowest value
print(scores * 1.1)      # applies the operation to every element at once
```

> **TIP**
> Notice there's no loop needed — NumPy applies operations to whole arrays at once. This is called vectorization, and it's dramatically faster than looping manually.

Lesson 11.2 pandas Basics

pandas builds on NumPy to give you spreadsheet-like tables (DataFrames) — the standard tool for real-world data work. Install with pip install pandas.

```
import pandas as pd

data = {
    "name": ["Amina", "John", "Sarah"],
    "score": [92, 85, 78]
}
df = pd.DataFrame(data)
print(df)
print(df["score"].mean())        # average score
print(df[df["score"] > 80])      # filter rows
```

Reading a real CSV file directly into a DataFrame:

```
df = pd.read_csv("students.csv")
print(df.head())          # first 5 rows
print(df.describe())      # quick statistical summary
```

🛠 TRY IT YOURSELF

Load students.csv (from Lesson 10.1) into a pandas DataFrame and print the average score, plus a filtered list of students who scored above 80.

Module 12 Arrays & Array Arrangement

Duration	3 lessons, ~3 hours
Goal	Build, reshape, sort, search, and stack arrays confidently

Lesson 12.1 Lists vs. Arrays: What's the Difference?

Python's built-in list is flexible but slow for heavy number-crunching. NumPy's array is faster, uses less memory, and supports whole-array math in one line. Install it with: pip install numpy

```
import numpy as np

py_list = [1, 2, 3, 4, 5]
np_array = np.array([1, 2, 3, 4, 5])

print(np_array * 2)     # [2 4 6 8 10] — works on every element instantly
# py_list * 2 would instead DUPLICATE the list: [1,2,3,4,5,1,2,3,4,5]
```

> **⚠ COMMON MISTAKE**
> Multiplying a Python list doesn't do math — it repeats the list. This trips up almost every beginner moving from lists to arrays for the first time.

Lesson 12.2 Creating & Arranging Arrays

```
import numpy as np

a = np.array([5, 2, 9, 1, 7])
zeros = np.zeros(5)              # [0. 0. 0. 0. 0.]
ones = np.ones((2, 3))           # 2 rows, 3 columns of 1s
sequence = np.arange(0, 10, 2)   # [0 2 4 6 8]
evenly = np.linspace(0, 1, 5)    # 5 evenly spaced numbers from 0 to 1
```

Arranging (sorting) an array puts its values in order:

```
a = np.array([5, 2, 9, 1, 7])
print(np.sort(a))          # [1 2 5 7 9]  ascending
print(np.sort(a)[::-1])    # [9 7 5 2 1]  descending (reverse the sorted result)
print(np.argsort(a))       # index positions that WOULD sort the array
```

Lesson 12.3 Reshaping, Stacking & Searching Arrays

```
a = np.arange(1, 13)              # [1 2 3 ... 12]
grid = a.reshape(3, 4)            # rearrange into 3 rows, 4 columns
print(grid)

flat = grid.flatten()             # back to a single 1-D array
print(flat)
```

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print(np.concatenate([x, y]))     # [1 2 3 4 5 6] — join end to end
print(np.vstack([x, y]))          # stack as two rows (vertical)
print(np.hstack([x, y]))          # stack side by side (horizontal)
```

Searching inside arrays without writing a loop:

```
a = np.array([10, 25, 3, 47, 25])
print(np.where(a == 25))          # index positions where value is 25
print(a[a > 20])                  # filter: only values greater than 20
print(np.max(a), np.min(a))       # 47 3
print(np.unique(a))               # [ 3 10 25 47] — duplicates removed
```

> **TIP**
> a[a > 20] is called boolean indexing — one of the most powerful and commonly used NumPy patterns. Read it as: "give me only the values of a where the condition is true."

> **TRY IT YOURSELF**
> Create an array of 15 random exam scores between 40 and 100 (np.random.randint(40, 100, 15)). Sort it, print the top 3 scores, and print how many scores are above 60 using boolean indexing.

Module 12 Mini-Project: Class Score Sorter

Build a program that:

1. Stores 10 student scores in a NumPy array.
2. Sorts them from highest to lowest.
3. Prints the top 3, the bottom 3, and the class average.
4. Prints how many students scored above the average using boolean indexing.

Module 13 Advanced Data Analysis

Duration	4 lessons, ~5 hours
Goal	Clean, group, merge, and visualize real datasets like a working analyst

Sample Datasets Download & Use These

Practice on real, free, public datasets. You can either download the CSV to your project folder, or load it straight from the link using pd.read_csv(url); both work identically.

Tips dataset (restaurant tips): *Small, beginner-friendly; great for your first analysis*
https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv

Titanic dataset (passenger survival): *Classic dataset for cleaning and grouping practice*
https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv

Iris dataset (flower measurements): *Good for basic statistics and simple visualization*
https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv

Kaggle Datasets (huge public catalog): *Search any topic; sales, sports, finance, health; free account needed*
https://www.kaggle.com/datasets

UCI Machine Learning Repository: *Long-standing academic source of clean, well-documented datasets*

https://archive.ics.uci.edu/

data.gov (US Government Open Data): *Real government datasets; population, economy, transport, and more*

https://data.gov/

```
import pandas as pd

# Load directly from the internet — no download needed
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
df = pd.read_csv(url)
print(df.head())
```

Lesson 13.1 Cleaning Real Data

Real data is almost never perfectly clean. These are the checks you run first, every time:

```
print(df.info())              # column types and non-null counts
print(df.isnull().sum())      # how many missing values per column
print(df.duplicated().sum())  # how many duplicate rows

df = df.dropna()                          # remove rows with missing data
```

```
# OR fill missing values instead of dropping them:
df["column"] = df["column"].fillna(df["column"].mean())

df = df.drop_duplicates()                # remove duplicate rows
df["price"] = df["price"].astype(float)  # fix a wrongly-typed column
```

> **⚠ COMMON MISTAKE**
> Dropping every row with any missing value (dropna()) can silently delete a large chunk of your dataset. Always check df.shape before and after to see how much data you actually lost.

Lesson 13.2 Grouping & Aggregating

groupby() is the single most important skill for real analysis — it answers questions like "average tip per day" in one line:

```
avg_by_day = df.groupby("day")["tip"].mean()
print(avg_by_day)

summary = df.groupby("day").agg(
    average_tip=("tip", "mean"),
    total_bills=("total_bill", "sum"),
    count=("tip", "count")
)
print(summary)
```

Lesson 13.3 Merging & Combining Data

Real-world data is usually split across multiple tables. merge() joins them together using a shared column, just like a database join:

```
orders = pd.DataFrame({"order_id": [1, 2, 3], "customer_id": [101, 102, 101]})
customers = pd.DataFrame({"customer_id": [101, 102], "name": ["Amina", "John"]})

merged = pd.merge(orders, customers, on="customer_id")
print(merged)
```

Lesson 13.4 Visualizing Data

Install with: pip install matplotlib. A chart communicates a trend faster than any table of numbers.

```
import matplotlib.pyplot as plt

avg_by_day = df.groupby("day")["tip"].mean()

avg_by_day.plot(kind="bar", color="teal")
plt.title("Average Tip by Day")
plt.xlabel("Day")
plt.ylabel("Average Tip ($)")
plt.tight_layout()
plt.savefig("average_tip_by_day.png")   # saves the chart as an image file
plt.show()                              # displays it on screen
```

> **TIP**
> Always label your axes and give the chart a title — an unlabeled chart is far less useful to whoever reads your report, including future you.

> **TRY IT YOURSELF**
> Using the Titanic dataset, group passengers by class (pclass) and calculate the survival rate per class (mean of the survived column). Plot it as a bar chart.

Module 13 Mini-Project: Restaurant Tips Analysis Report

Using the tips.csv dataset:

1. Load the data and clean it (check for missing values and duplicates).
2. Calculate the average tip percentage (tip ÷ total_bill × 100) overall and by day.
3. Find which day and time (lunch/dinner) generates the highest average bill.
4. Create two charts: average tip by day, and total bill distribution.
5. Write 3–5 sentences summarizing your findings, like a real analyst report.

Module 14 Web Development with Python (Flask)

Duration	4 lessons, ~4 hours
Goal	Build and run a real, working website

Lesson 14.1 Your First Flask App

Flask is a lightweight web framework perfect for beginners because it stays simple while still being used in real production projects. Install with: pip install flask

```
# app.py
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Welcome to my first website!</h1>"

if __name__ == "__main__":
    app.run(debug=True)
```

Run it with python app.py, then open http://127.0.0.1:5000 in your browser.

> **TIP**
> debug=True automatically reloads your site when you save changes, and shows helpful error pages in the browser while you're building.

Lesson 14.2 Routes & Multiple Pages

```
@app.route("/")
def home():
    return "<h1>Home Page</h1>"

@app.route("/about")
def about():
    return "<h1>About Us</h1><p>We teach practical Python skills.</p>"

@app.route("/contact")
def contact():
    return "<h1>Contact</h1><p>Email: info@example.com</p>"
```

Routes can also accept parts of the URL as variables:

```
@app.route("/user/<name>")
def greet_user(name):
    return f"<h1>Hello, {name}!</h1>"
# Visiting /user/Amina shows "Hello, Amina!"
```

Lesson 14.3: Templates With Jinja

Writing HTML inside return strings gets messy fast. Flask uses Jinja templates instead — separate .html files that can include Python-like logic.

Create a folder called templates, and inside it a file home.html:

```
<!-- templates/home.html -->
<!DOCTYPE html>
<html>
<head><title>{{ page_title }}</title></head>
<body>
    <h1>Welcome, {{ user_name }}!</h1>
    <ul>
    {% for skill in skills %}
        <li>{{ skill }}</li>
    {% endfor %}
    </ul>
</body>
</html>
```

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route("/")
def home():
    return render_template(
        "home.html",
        page_title="My Site",
        user_name="Amina",
        skills=["Python", "Flask", "Data Analysis"]
    )
```

> **⚠ COMMON MISTAKE**
> Flask looks for templates only inside a folder literally named templates in the same directory as app.py — a very common beginner setup mistake is placing HTML files elsewhere.

Lesson 14.4 Handling Forms

```
from flask import Flask, render_template, request
app = Flask(__name__)

@app.route("/contact", methods=["GET", "POST"])
def contact():
    if request.method == "POST":
        name = request.form["name"]
        message = request.form["message"]
        with open("messages.txt", "a") as f:
            f.write(f"{name}: {message}\n")
        return "<h2>Thank you! Your message was received.</h2>"
    return render_template("contact.html")
```

```
<!-- templates/contact.html -->
<form method="POST">
    <input type="text" name="name" placeholder="Your name"><br>
    <textarea name="message" placeholder="Your message"></textarea><br>
    <button type="submit">Send</button>
</form>
```

> **🛠 TRY IT YOURSELF**
> Add a /projects route to your Flask app that displays a list of 3 fake project names using a Jinja {% for %} loop in a template.

Module 14 Mini-Project: Mini Personal Website

Build a 3-page Flask site: Home (introduces you), Skills (a Jinja loop over a Python list of your skills), and Contact (a working form that saves messages to a text file). Use one shared layout via Jinja template inheritance (a base.html with a navigation bar the other pages extend).

Module 15 Simple Projects Practice Bank

Duration	Ongoing; pick 1–2 per week
Goal	Reinforce every module with short, focused, achievable builds

These are intentionally small; most take 20–60 minutes. Use them between modules whenever you want extra repetition before moving on, or as warm-ups before a training session.

Foundations & Logic (Modules 1–3)

1. Temperature Converter — convert Celsius to Fahrenheit and back.
2. BMI Calculator — take height and weight, calculate and classify BMI.
3. Even/Odd Checker — tell the user if a number is even or odd.
4. Multiplication Table Generator — print the times table for any number 1–12.
5. Simple ATM Simulator — deposit, withdraw, and check balance using a while loop menu.

Data Structures & Functions (Modules 4–5)

1. Shopping List Manager — add, remove, and view items in a list.
2. Word Frequency Counter — count how often each word appears in a sentence using a dictionary.
3. Unique Visitor Tracker — use a set to track unique usernames from a list with duplicates.
4. Tip Calculator Function — a reusable function that takes a bill and tip percentage.
5. Rock, Paper, Scissors — play against the computer using random.choice().

Strings, Files & OOP (Modules 6–8)

1. Palindrome Checker — check if a word reads the same backward.
2. Password Strength Checker — flag weak passwords (length, digits, symbols).
3. Simple Diary App — append dated entries to a text file.
4. Contact Class — a Contact class with name/phone/email and a display() method.
5. Vehicle Inheritance Demo — a Vehicle base class with Car and Motorbike subclasses.

Arrays & Data Analysis (Modules 12–13)

1. Grade Statistics — use NumPy to find mean, median, highest, and lowest of a score array.
2. Matrix Builder — create and reshape a 3x3 array, then find its transpose.
3. CSV Explorer — load any CSV with pandas and print its shape, columns, and summary stats.
4. Sales Bar Chart — group a small sales dataset by category and plot it.

Web Development (Module 14)

1. Digital Business Card — a one-page Flask site showing your name, role, and links.
2. Quote of the Day — a Flask route that shows a random quote from a Python list each time the page loads.
3. Simple Feedback Form — a form that saves visitor feedback to a JSON file.

> **TIP**
> Rebuilding the same idea in two different modules (e.g. a Contact Book as a dictionary in Module 4, then as a class in Module 8) is one of the fastest ways to feel real progress; the code gets cleaner as your skills grow.

Module 16 Capstone Projects

Duration	5 projects, 3–8 hours each
Goal	Combine everything learned into complete, portfolio-ready programs

These capstones intentionally don't give you exact code; that's the point. Use everything from Modules 0–15. Break each project into small steps, build one piece at a time, and test constantly.

Capstone 1 — Command-Line To-Do List Manager

Combines: functions, lists/dictionaries, loops, file/JSON handling, error handling.

- Menu-driven program: Add task, Complete task, Delete task, View all tasks, Exit.
- Each task should have a description, a priority (low/medium/high), and a completed status.
- Tasks persist between runs by saving/loading from a JSON file.
- Handle invalid menu choices without crashing.
- Stretch goal: sort tasks by priority when displaying them.

Capstone 2 Student Grade Management System

Combines: OOP (classes), CSV/file handling, functions, data structures.

- A Student class with name, and a list of scores.
- A GradeBook class that holds multiple Student objects.
- Methods to add a student, add a score, calculate each student's average, and calculate the class average.
- Load starting data from a CSV, and export a summary report to a new CSV.
- Stretch goal: rank students from highest to lowest average.

Capstone 3 Weather Dashboard (API Project)

Combines: APIs (requests), JSON, functions, error handling, formatting output.

- Sign up for a free API key at openweathermap.org (or a similar free weather API).
- Ask the user for a city name, call the API, and display temperature, conditions, and humidity.
- Handle a city that doesn't exist (bad API response) gracefully.
- Let the user check multiple cities in one session using a loop, until they choose to exit.
- Stretch goal: save each search to a JSON "history" file with a timestamp.

Capstone 4 Sales Data Analysis Report

Combines: pandas, NumPy, arrays, data cleaning, visualization (Modules 12–13).

- Use the sample sales dataset from Module 13 (or your own CSV).
- Clean the data: handle missing values and incorrect types.
- Calculate total revenue, best-selling product, and monthly trend.
- Produce at least two charts (bar chart of sales by category, line chart of sales over time).
- Export a clean summary table to a new CSV or Excel file.

Capstone 5 Personal Portfolio Website (Flask)

Combines: Web Development (Module 14), functions, file handling, HTML basics.

- Build a small Flask site with at least 3 pages: Home, Projects, Contact.
- Use one shared base template with a navigation bar (Jinja template inheritance).
- Display your capstone projects from this course on the Projects page, pulled from a Python list or JSON file rather than hardcoded HTML.
- Add a simple contact form that saves submitted messages to a file.
- Stretch goal: deploy it for free on a platform like Render or PythonAnywhere.

> **TIP**
> Push all five capstones to a free GitHub account once you're done. This becomes your first real portfolio; exactly what employers and clients ask to see.

Module 17 Pro Habits & Next Steps

Duration	3 lessons, ~3 hours
Goal	Build habits that separate hobbyists from professionals

Lesson 17.1 Debugging Like a Pro

1. Read the error message from the bottom up — the last line usually names the actual problem (e.g., NameError, TypeError).
2. Find the line number mentioned in the traceback — that's exactly where to start looking.
3. Use print() statements to check the value and type of variables at different points — this is the single most effective beginner debugging tool.
4. Isolate the problem: comment out sections of code to narrow down which part is actually failing.
5. Search the exact error message online — virtually every Python error has already been asked and answered by someone else.

Lesson 17.2 Writing Clean, Professional Code

- Follow PEP 8 (Python's official style guide): 4-space indentation, lowercase_with_underscores for variables and functions, PascalCase for class names.
- Write comments that explain why, not what; the code itself already shows what it does.
- Keep functions short and focused on doing one thing well.
- Use meaningful names: calculate_total_price() beats calc() or do_thing().
- Avoid repeating yourself (DRY principle) — if you copy-paste code three times, it probably belongs in a function.

Lesson 17.3 Version Control with Git (Essential Basics)

Git tracks changes to your code over time, and GitHub hosts your projects online for others to see.

```
git init                          # start tracking a project
git add .                         # stage all changed files
git commit -m "Add grade calculator"   # save a snapshot with a message
git push                          # upload to GitHub
```

> **TIP**
> Commit often, with clear messages. Your commit history becomes a visible record of your learning progress — and professionals check it.

Lesson 17.4 Essential Command Reference

A working command-line vocabulary is a professional requirement, not an extra. These are the commands you'll use in almost every project.

Terminal / File Navigation Commands

Command	What It Does
cd folder_name	Move into a folder (cd .. moves up one level)
dir (Windows) / ls (Mac/Linux)	List files in the current folder
mkdir folder_name	Create a new folder
python file.py	Run a Python file
python or python3	Open the interactive Python shell (REPL)
exit()	Leave the interactive Python shell
clear (Mac/Linux) / cls (Windows)	Clear the terminal screen

pip (Package Manager) Commands

Command	What It Does
pip install package_name	Install a library, e.g. pip install pandas
pip uninstall package_name	Remove an installed library
pip list	Show all installed packages in the current environment
pip freeze > requirements.txt	Save all installed packages to a file to share with others
pip install -r requirements.txt	Install every package listed in that file

Virtual Environment Commands

A virtual environment keeps each project's packages separate, so projects never conflict with each other.

```
python -m venv venv          # create an environment named "venv"
venv\Scripts\activate       # activate it on Windows
source venv/bin/activate     # activate it on Mac/Linux
deactivate                   # turn it off when you're done
```

Git Quick Reference

Command	What It Does
git status	Show what's changed since your last commit
git log	Show commit history
git clone url	Copy a project from GitHub to your computer
git pull	Download the latest changes from GitHub
git branch new-feature	Create a new branch to work on safely
git checkout branch-name	Switch to a different branch

> **TIP**
> Type any command with --help after it (e.g. pip --help) to see its full list of options directly in the terminal.

Lesson 17.5 Where to Go From Here

Path	Next Steps
Web Development	Deepen Flask, then learn Django, HTML/CSS/JavaScript, and deployment
Data Analysis / Science	Deepen pandas & NumPy, learn Matplotlib/Seaborn, then scikit-learn
Automation / Scripting	Learn os, shutil, schedule, and browser automation with Selenium
Software Engineering	Learn testing (pytest), design patterns, and contribute to open source
General	Solve daily problems on sites like Codewars, LeetCode, or Exercism

Quick Reference Cheat Sheet

Syntax at a Glance

Concept	Syntax
Variable	name = value
If statement	if condition:
For loop	for item in sequence:
While loop	while condition:
Function	def name(params):
Class	class Name:
Try/except	try: ... except ErrorType:
Import	import module_name
List	my_list = [1, 2, 3]
Dictionary	my_dict = {"key": "value"}
NumPy array	np.array([1, 2, 3])
f-string	f"Value is {variable}"

Common Errors & What They Mean

Error	Usual Cause
SyntaxError	Typo, missing colon, or mismatched brackets/quotes
NameError	Using a variable that doesn't exist yet (check spelling)
TypeError	Mixing incompatible types, e.g. "5" + 5
IndexError	Accessing a list/array position that doesn't exist
KeyError	Accessing a dictionary key that doesn't exist
IndentationError	Inconsistent spacing/tabs — stick to 4 spaces
ZeroDivisionError	Dividing a number by zero
ModuleNotFoundError	Library not installed — run pip install package_name

Final Notes for Trainers & Learners

For Trainers

- Each module is designed as one training session (2–4 hours) or split across 2 shorter sessions.
- Live-code every example in front of learners rather than just showing finished code; watching mistakes get fixed is where real learning happens.
- Insist learners complete every "Try It Yourself" and mini-project before moving to the next module. Speed without practice creates gaps that compound.
- Review capstone projects individually and give specific, code-level feedback, not just "good job."

For Self-Learners

- Consistency beats intensity; 45 minutes daily will outperform one long session per week.
- Struggling with a bug for 20–30 minutes before looking up the answer is normal and valuable. That struggle is where understanding is actually built.
- Don't skip the mini-projects. Reading code and writing code from scratch are very different skills, and only one of them makes you a programmer.
- Revisit Module 1–3 concepts after finishing Module 8 — they'll make even more sense once you understand what they're building toward.

You now have a complete, practical path from zero to a confident, job-capable Python foundation covering core programming, data analysis, and web development. The only ingredient left is consistent practice; good luck, and enjoy the process.

Bright Emerald Computer Academy

Trainer / Tutor: Emmanuel Sebit Joseph

Empowering learners with practical, job-ready computer skills.